

Solving Multiphysics and Multiscale Problems with AI-Augmented Simulations (CFD-PINN)

Part 3: Inside the Toolbox, Activations, Optimisers, and Losses That Train

Jeff Gomes and Henglai Zhan

(Twitter/X: @CoPhyMs)



XXIX ENMC
ENCONTRO NACIONAL DE MODELAGEM COMPUTACIONAL

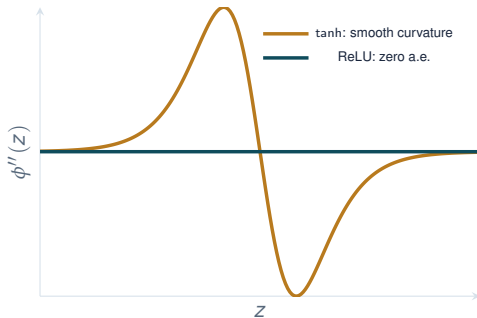
XVII ECTM
ENCONTRO DE CIÊNCIA E TECNOLOGIA DE MATERIAS

Section Outline

- 1 Activation Functions
- 2 FEM-PINN: Deep Ritz and Deep Galerkin methods
- 3 Optimisation
- 4 Loss Functions
- 5 FEM-PINN Porous Media Problem
- 6 Conclusion

Why Does tanh Dominate PINN Architectures?

- **Standard DL (ReLU):** Computationally cheap and avoids vanishing gradients, but piecewise linear.
- **PINN Requirement:** PDE residual losses demand high-order input derivatives (e.g., u_{xx} or u_{tttt}).
- **The ReLU Bottleneck:** Second-order derivative which fails to convey curvature to the optimiser.
- **Smooth Alternatives:** Smooth functions (tanh, sin, GELU) have well-behaved derivatives of *every* order, which is why tanh became the PINN community's default.



Adaptive Activation Functions

- In addition to an overview of linear transformations (weights), [Jagtap et al. \(2020\)](#) also investigated *nonlinear* transformations.

- A **global adaptive activation**,

$$\sigma(z) = \tanh(\alpha z),$$

introduces a single trainable scalar α that rescales the pre-activation function.

- α is a single trainable parameter, shared across the whole network (or one per layer, for a layer-wise variant).: $\phi(\alpha z)$.
- Learning α effectively learns the **slope** of the activation function, adapting the network's expressivity during training.
- Empirically, this accelerates convergence and can improve final accuracy, at negligible extra parameter cost.

Section Outline

- 1 Activation Functions
- 2 FEM-PINN: Deep Ritz and Deep Galerkin methods**
- 3 Optimisation
- 4 Loss Functions
- 5 FEM-PINN Porous Media Problem
- 6 Conclusion

Brief Review of Ritz & Galerkin Methods

Galerkin Method

Rather than minimising a variational functional, the Galerkin method projects the strong-form PDE residual $R(u_h)$ onto a finite-dimensional test space V_h . By requiring

$$\int_{\Omega} R(u_h) v_h d\Omega = 0 \quad \forall v_h \in V_h,$$

it enforces orthogonality between the residual error and the approximating space.

Ritz Method

The classical Rayleigh-Ritz method casts a self-adjoint boundary value problem as the minimisation of a potential energy functional over an admissible space of functions. By restricting this search to a finite-dimensional trial space, the necessary stationarity condition ($\delta\Pi = 0$) recovers the Euler-Lagrange equation in its discrete weak form.

Both are classical FEM and pick a finite-dimensional trial/test space (traditionally, piecewise polynomials on a mesh). Now, we will review the deep-learning generalisation of each: swap the trial space for a neural network.

The Deep Galerkin Method (DGM)

- Sirignano & Spiliopoulos (2017) replace the finite-dimensional trial space with a neural network u_θ .
- Minimise the **strong-form residual** directly, approximated by **Monte Carlo** sampling of collocation points

$$\text{Strong Form: } \mathbb{E}_x \left[|\mathcal{N}[u_\theta](x) - f(x)|^2 \right],$$

(the same as the [vanilla PINN recipe](#) from Seminar 1).

- No explicit test function space is defined. The residual is simply driven to zero pointwise, in expectation over the sampled points.
- DGM was proposed independently of the PINN formulation.

The Deep Ritz Method (DRM)

- The Deep Ritz Approach (E & Yu, 2017) parameterises the trial space using a deep neural network and minimises the continuous energy functional.
- **Natural Weak Form:** Euler-Lagrange equations implicitly satisfy the weak (energy) form,

$$\min_{\theta} \mathcal{E}[u_{\theta}].$$

Integration by parts is directly embedded in the objective function.

- **Variational Alignment:** Naturally suited for PDEs derived from energy minimisation (*e.g.*, elasticity, potential flow, spectral problems).
- **Reduced Regularity:** Requires lower-order derivatives than DGM, improving stability and computational speed per training iteration.

Variational PINNs (vPINN)

- [Kharazmi, Zhang & Karniadakis \(2019\)](#) proposed a method almost identical to DGM/DRM, but with an **explicit test-function space** chosen for the weak (variational) formulation.
- This is a genuine **Petrov-Galerkin** scheme: trial functions (the network) and test functions (chosen basis, *e.g.*, polynomials) can differ.
- The loss becomes minimising a bilinear form evaluated against each test function, much closer in spirit to classical FEM than DGM's pure Monte Carlo residual.

THREE FORMULATIONS

DGM/PINN vs. DRM/PINN vs. vPINN

DGM / PINN

- Strong-form residual minimisation;
- Monte Carlo loss approximation;
- No explicit test space.

DRM / PINN

- “Energy” minimisation (integrated-by-parts PDE);
- Weak form emerges as the Euler-Lagrange equation.

vPINN

Variational/Petrov–Galerkin: choose a test basis, minimise the bilinear form on it.

All three = neural trial space + a numerical formulation (strong, energy, or variational) + optimisation. They differ in the functional.

Section Outline

- 1 Activation Functions
- 2 FEM-PINN: Deep Ritz and Deep Galerkin methods
- 3 Optimisation**
- 4 Loss Functions
- 5 FEM-PINN Porous Media Problem
- 6 Conclusion

RECAP

First-Order Methods and Their Limits

- Stochastic Gradient Descent (SGD) and Adaptive Moment Estimation (Adam) use only **first-order** information (*i.e.*, gradient) to update parameters.
- They are cheap per iteration and scale well, but PINN loss landscapes are often **ill-conditioned**: narrow, curved valleys where a fixed step size is either too timid or too aggressive.
- Near a good minimum, first-order methods can stall, oscillate, or converge very slowly along flat directions.
- This motivates bringing in **curvature** (second-order) information.

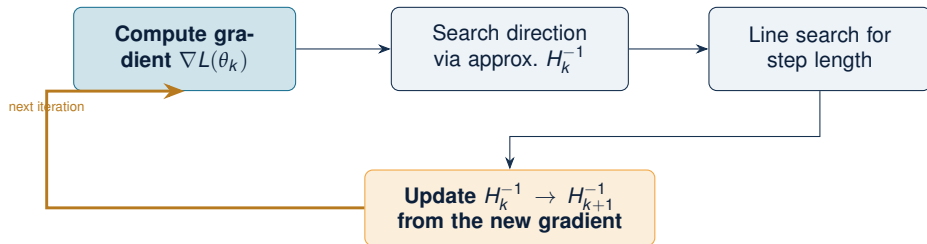
A QUASI-NEWTON METHOD

Optimisation: L-BFGS

- Newton's method uses the exact inverse Hessian (H) to take curvature-aware steps, but computing and inverting the Hessian is prohibitively expensive for a neural network.
- The **Broyden-Fletcher-Goldfar-Shanno (BFGS)** builds up an *approximation* to the inverse Hessian iteratively, using only gradient information from previous steps.
- It stores a full $n \times n$ approximation of the inverse Hessian matrix.
- **L-BFGS** ("Limited-memory" BFGS) stores only a small fixed number (m) of recent correction vectors implicitly, instead of a full dense matrix, making it tractable for networks with thousands to millions of parameters.
- It steers the search through parameter space using this estimated curvature, converging faster than first-order methods *near* a minimum.

SCHEMATIC

The BFGS Update, Sketched



L-BFGS keeps only the last m (typically 5–20) gradient/update pairs, approximating H^{-1} without ever forming it explicitly.

PRACTICAL RECIPE

Two-Stage Optimisation: Adam then L-BFGS

Stage 1: Adam

Fast, robust, coarse descent from a random

Stage 2: L-BFGS

Precise, curvature-aware refinement near the minimum



This is now close to a de-facto standard PINN training recipe: Adam gets you into the right basin, L-BFGS polishes the solution once there.

Section Outline

- 1 Activation Functions
- 2 FEM-PINN: Deep Ritz and Deep Galerkin methods
- 3 Optimisation
- 4 Loss Functions**
- 5 FEM-PINN Porous Media Problem
- 6 Conclusion

LOSS FUNCTIONS

The standard PINN loss

- Every PINN loss so far has been a weighted sum: PDE residual + boundary + initial conditions (+ data, if available).
- Each term is a mean-squared error, treating every sampled point, and every moment in time, as equally important.
- But physically, this is often *not* true: some regions of space (sharp gradients, corners) and some points in time (early transients) matter more for accurate training.
- The rest of this part builds losses that respect this uneven importance.

Loss Function

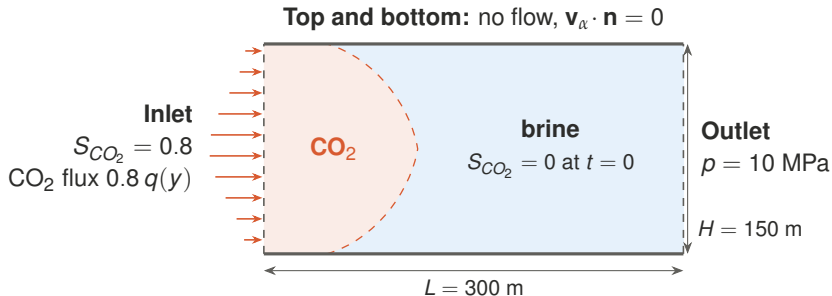
$$L = \lambda_r L_{\text{PDE}} + \lambda_b L_{\text{BC}} + \lambda_0 L_{\text{IC}}$$

Scalar weights, uniform importance within each term.

Section Outline

- 1 Activation Functions
- 2 FEM-PINN: Deep Ritz and Deep Galerkin methods
- 3 Optimisation
- 4 Loss Functions
- 5 FEM-PINN Porous Media Problem**
- 6 Conclusion

Example 3: CO₂ Displacing Brine



Two-phase Darcy flow, $\alpha \in \{b, c\}$

$$\phi \partial_t S_\alpha + \nabla \cdot \mathbf{v}_\alpha = 0, \quad S_b + S_c = 1$$

$$\mathbf{v}_\alpha = -\frac{K k_{r\alpha}}{\mu_\alpha} \nabla p$$

Corey: $k_{rc} = \hat{S}^2$, $k_{rb} = (1 - \hat{S})^2$,
 with $\hat{S} = S_c/0.8$

Parameters

$$\phi = 0.2, \quad K = 10^{-12} \text{ m}^2, \quad t \in [0, 300] \text{ days}$$

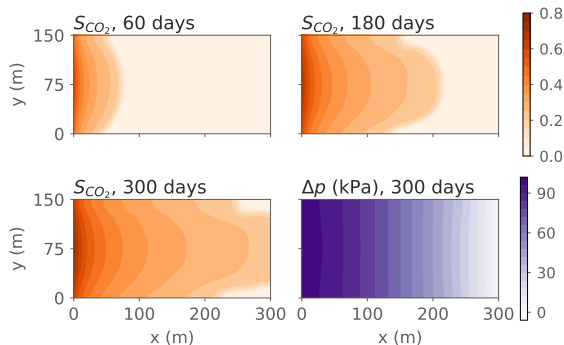
$$\mu_b = 10^{-3}, \quad \mu_c = 7 \times 10^{-5} \text{ Pa s}$$

$$\text{Mobility ratio } M = \mu_b / \mu_c \approx 14 \text{ (adverse)}$$

$$q(y) = q_0 \left[1 + \frac{1}{2} \cos 2\pi \left(\frac{y}{H} - \frac{1}{2} \right) \right], \quad q_0 = 10^{-6} \text{ m/s}$$

Neglected: gravity, capillarity, dissolution, compressibility, heterogeneity (teaching benchmark).

Reference solution: ICFERST



- 61×31 -vertex triangular mesh, $\Delta t = 1$ day, 16 snapshots
- Curved front: inflow $1.5 q_0$ at mid-height vs $0.5 q_0$ at the walls; breakthrough ≈ 260 days
- Mean S_{CO_2} tracks the injected volume ($+0.023$ per 20 days) until breakthrough
- Δp falls $233 \rightarrow 100$ kPa as mobile CO₂ replaces brine

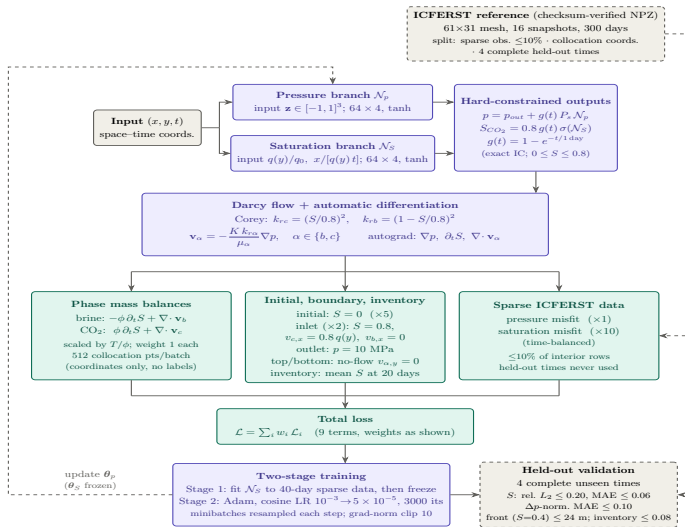
Role in the PINN study

Train: sparse labels ($\leq 10\%$ of rows)

Physics: unlabelled collocation points

Score: unseen 20, 100, 200, 300 d

Reservoir Simulator + PINN



Section Outline

- 1 Activation Functions
- 2 FEM-PINN: Deep Ritz and Deep Galerkin methods
- 3 Optimisation
- 4 Loss Functions
- 5 FEM-PINN Porous Media Problem
- 6 Conclusion**

SUMMARY

Key takeaways — across all three lectures

- **Lecture 1:** PINNs embed a PDE residual, built from automatic differentiation, directly into a neural network's loss: turning a universal function approximator into a differential-equation solver.
- **Lecture 2:** PINNs are not free lunches, convergence theory, the Information Bottleneck, and gradient-conflict pathologies explain *why* vanilla training can fail, and motivate the advanced schemes that fix it.
- **Lecture 3:** Activation choice, the FEM/variational connection, second-order optimisation, and multi-scale loss weighting are the practical levers that turn a fragile prototype into a trainable, production-grade PINN.
- **Throughout:** PINNs sit at the intersection of functional analysis, numerical PDEs, and deep learning, multiphysics and multiscale simulation augmented by, not replaced by, AI.